

Class 6 : R Functions

Kaif Yenepoya PID (A17497464)

Table of contents

Background	1
A first function	1
A generate_dna() function	3
Generate random protein sequences of length 6 to 13	6
ARE OUR OPEPTIDES “UNIQUE IN NATURE” ?	6
Connecting our findings to immunology	7

Background

All functions in R have at least 3 things:

- a *name* (we pick and use it to call the function)
- input *arguments* (*one or more comma seperated inputs that go inside the brackets when we call the function*), -the *body** (the lines of R code that do work of the function)

A first function

Here we will create a function to add some numbers. Let’s call it ‘add()’

Input arguments can either be “required” or “optional”. The later have fall-back default values that will be used if the user does not specify them.

Q1a. Your first version of the function should add two input numbers together.
For example, add(4, 7) should return 11. [1 pt]

```
add <- function (x, y=0){  
  x + y  
}
```

Can we use our new function:

```
add (10, 100)
```

```
[1] 110
```

```
add(10)
```

```
[1] 10
```

Q1b : For your second version, adapt your first function so it can take a single input vector or two inputs as before. For example, `add(4, 7)` and `add( c(4,7,10) )`.
[1 pt]

```
add <- function(x, y = NULL) {  
  # If y is not given, assume x is a vector and sum all values  
  if (is.null(y)) {  
    return(sum(x))  
  }  
  
  # If y is given, add x and y  
  return(x + y)  
}
```

```
add(4, 7)
```

```
[1] 11
```

```
add(c(4, 7, 10))
```

```
[1] 21
```

Q1c: Finally, on your own (outside of class) create a third version of your function that can add any number of inputs that the user provides. For example, `add(1, 2, 3, -4)` should return 2. Hint: explore the `...` (dots) argument or the base R function `sum()`. [2 pts]

```
add <- function(...) {
  # Collect all inputs into a list
  inputs <- list(...)

  # Unlist converts all inputs into one numeric vector, then sum adds them
  return(sum(unlist(inputs)))
}
```

```
add(4, 7)
```

```
[1] 11
```

```
add(c(4, 7, 10))
```

```
[1] 21
```

```
add(1, 2, 3, -4)
```

```
[1] 2
```

We can explicitly set a **return** value output from a function (rather than the default last line) by using the ‘return()’ function call.

A generate_dna() function

A useful function here is the “base R” ‘sample()’ function

```
sample(x=c("A", "C", "G", "T"), size=10, replace =TRUE)
```

```
[1] "C" "C" "C" "T" "A" "C" "C" "C" "G" "G"
```

We can use this to make a random nucleotide sequence if we draw from “A”, “C”, “G” and “T” ...

Q2a: Write a function generate_dna() that returns a random DNA sequence of a length specified by the user. a. Your first version should return a multi-element vector of single character nucleotides. For example generate_dna(6) might return “A”, “T”, “T”, “G”, “A”, “C”. [1 pt]

```
generate_dna <- function(len=10) {
  sample(x=c("A", "C", "G", "T"), size=len, replace =TRUE)
}
```

```
generate_dna()
```

```
[1] "T" "G" "G" "A" "C" "C" "A" "A" "A" "C"
```

```
generate_dna(len=100)
```

```
[1] "T" "A" "T" "C" "T" "G" "C" "G" "T" "T" "G" "A" "G" "C" "A" "T" "C" "G"
[19] "C" "T" "G" "A" "T" "T" "C" "C" "C" "C" "A" "C" "G" "A" "G" "G" "T" "A"
[37] "C" "T" "G" "G" "T" "G" "T" "C" "A" "C" "A" "T" "G" "G" "G" "G" "A" "G"
[55] "G" "G" "G" "A" "C" "T" "T" "A" "G" "T" "G" "C" "T" "A" "G" "G" "C" "A"
[73] "T" "C" "A" "G" "A" "T" "A" "A" "A" "T" "C" "C" "T" "T" "G" "T" "T" "G"
[91] "T" "G" "T" "T" "G" "A" "C" "G" "T" "C"
```

Q2b : b. Your second version should optionally be able to return either a multi-element vector of single character nucleotides (as before) or a single character string (not a vector of single letters but a single vector of multiple letters). For example “AAGGCTG”. [1 pt]

Functions that could be useful here are ‘paste()’, ‘if()’, ‘cat()’ and ‘return()’

```
generate_dna <- function(len = 10, as_string = FALSE) {
  dna <- sample(x = c("A", "C", "G", "T"), size = len, replace = TRUE)

  if (as_string) {
    return(paste(dna, collapse = ""))
  }

  return(dna)
}
```

```
generate_dna(10)
```

```
[1] "A" "G" "T" "G" "A" "A" "G" "A" "A" "G"
```

```
generate_dna(10, as_string = TRUE)
```

```
[1] "AGTCGTTAGA"
```

Q2C : c. Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length.

```
generate_dna <- function(len = 10) {  
  # Generate a random vector of DNA nucleotides  
  dna <- sample(x = c("A", "C", "G", "T"), size = len, replace = TRUE)  
  
  # Collapse the vector into one single DNA string  
  dna_string <- paste(dna, collapse = "")  
  
  # Print the result in FASTA format with a sequence length ID  
  cat(">len", len, "\n", dna_string, "\n", sep = "")  
}
```

```
generate_dna(9)
```

```
>len9  
GGGGGTACC
```

Q3. Write a function `generate_protein()` that returns a random peptide/protein sequence of a length specified by the user. For example `generate_protein(6)` might return "WQRTAG".

```
generate_protein <- function(len = 6) {  
  # Store the 20 standard amino acid one-letter codes  
  aa <- c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I",  
          "L", "K", "M", "F", "P", "S", "T", "W", "Y", "V")  
  
  # Randomly sample amino acids to make a protein sequence  
  protein <- sample(x = aa, size = len, replace = TRUE)  
  
  # Collapse the amino acid vector into one single string  
  protein_string <- paste(protein, collapse = "")  
  
  # Return the final protein sequence  
  return(protein_string)  
}
```

```
generate_protein(6)
```

```
[1] "SACKYA"
```

Generate random protein sequences of length 6 to 13

Q4. Adapt and use your `generate_protein()` function to generate a series of random protein sequences ranging from 6 to 13 amino acids in length (one sequence per length). Take advantage of the base R function `for()` or `sapply()` so that you do not have to call `generate_protein()` eight times by hand.

```
# Generate random protein sequences of length 6 to 13 in FASTA format
for (i in 6:13) {
  cat(">id.", i, "\n", generate_protein(i), "\n", sep = "")
}
```

```
>id.6
TWPCSV
>id.7
HPKRCDD
>id.8
LKTWEYHV
>id.9
CLQGMEHGF
>id.10
QHCNIPPKYG
>id.11
LNFSIWDKNED
>id.12
DGCGVWFDCHKE
>id.13
VVTRGDNYMLDSG
```

ARE OUR OPEPTIDES “UNIQUE IN NATURE” ?

Q5 : Take your FASTA-formatted peptides from Q4 and run them as a single BLASTp search against the Non-redundant protein sequences (nr) database at <https://blast.ncbi.nlm.nih.gov/>. For this question do not restrict the organism (leave the Organism field blank so that the full nr database is searched).

length	Ide	Cov	Unique
6	100	100	N
7	100	100	N
8	100	100	N
9	100	89	Y
10	100	90	Y
11	81.82	100	Y
12	88.89	75	Y
13	77.78	69	Y

Q5a : At which sequence length do your randomly generated peptides start to look “unique in nature” (i.e. no 100% coverage + 100% identity hit)? [1 pt]

The randomly generated peptides started to look unique in nature at length 9, because this was the first peptide without a hit showing both 100% query coverage and 100% identity.

Q5b : Speculate why very short random peptides are almost always found in nr, while longer ones typically are not. Your answer should refer both to the size of the sequence space (20^L for a peptide of length L) and to the size of the known protein universe. [1 pt]

The probability that very short peptides will be detected within nr is higher due to the relatively limited size of the sequence space. The exponential increase in the size of the sequence space, 20^L , will soon become much larger than the known size of the protein universe, decreasing the possibility of exact matches.

Connecting our findings to immunology

Q6 : In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice for the immune system to present very short peptides? [3 pt]

The lower limit of the length of peptides is estimated to be approximately nine amino acids since, as indicated by our Q5 results, peptides between six to eight amino acids retained 100% sequence match identity and coverage, whereas peptides above length nine became unique in their nature. The use of very short peptides for antigen recognition and presentation is a poor idea as there will be plenty of them in already known proteins, making them hard to discriminate between self and non-self.